Characterizing High Bandwidth Flash for LLM Serving 论文解读

Zack Yu、Chloe Wong、Coleman Hooper 等｜UC Berkeley、FuriosaAI 等｜arXiv:2609.39131v2, 2026-10-05

一、背景动机

1. HBM 与 HBF：容量大不等于直接替换

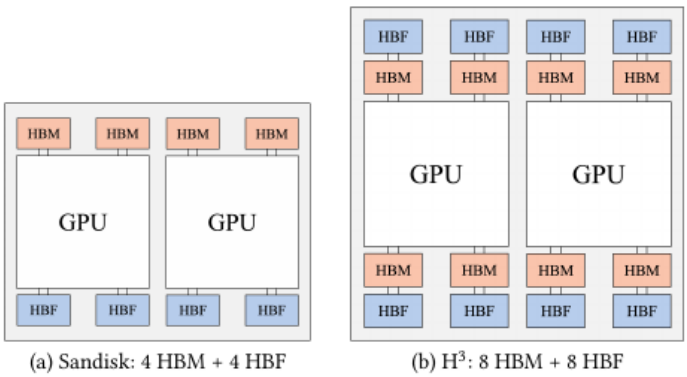
- **HBM 的优势与压力：** 访问快、适合频繁读写，但模型权重、活跃请求的 KV 和等待复用的历史 KV 会争抢容量。扩大 batch 可以摊薄权重读取，却也需要更多 KV 空间。
- **HBF 的机会：** 以堆叠 NAND 扩展加速器内存。本文假设单堆栈容量为 **375 GB**，约为 24 GB HBM 的 **15.6 倍**；两者读取带宽均为 **1 TB/s**。
- **HBF 的代价：** 读取启动延迟为 **20 µs**，HBM 为 0.1 µs；写入带宽仅 **8 GB/s**，且受擦写寿命限制。本文还假设其读取能耗为 HBM 的 **2 倍**，因此不能只看容量。[1]

2. 一种重要场景：多轮 Agent 高吞吐服务

- **前缀会反复使用：** 同一会话调用工具、接收结果、再次请求模型，后续输入会复用前轮上下文。保留 KV 可以避免从主机回载，甚至避免重复 prefill。
- **目标是整批任务完成得快：** 面向合成数据、异步 Agent、强化学习 rollout 等大量独立会话，允许单请求延迟增加；论文不设置单请求延迟约束。[2]

3. 核心问题：如何把容量收益变成性能与寿命收益

- **挑战 1（数据放置）：** 权重、活跃 KV、历史 KV 和稀疏注意力索引的访问频率不同，全部放入 HBF 会增加访问代价。
- **挑战 2（缓存抖动）：** 不断接纳新请求，可能把即将复用的历史 KV 挤出去，之后又重新加载和写入 HBF。
- **挑战 3（连接资源）：** HBM 与 HBF 若争用 GPU 周边接口，扩容会牺牲 HBM；H^3 保留 HBM 后再连接 HBF，但使用更多堆栈，成本也更高。[3]



原文图 2：左为 4 HBM+4 HBF；右为 H^3 的 8 HBM+8 HBF，数量均按每 GPU 计。

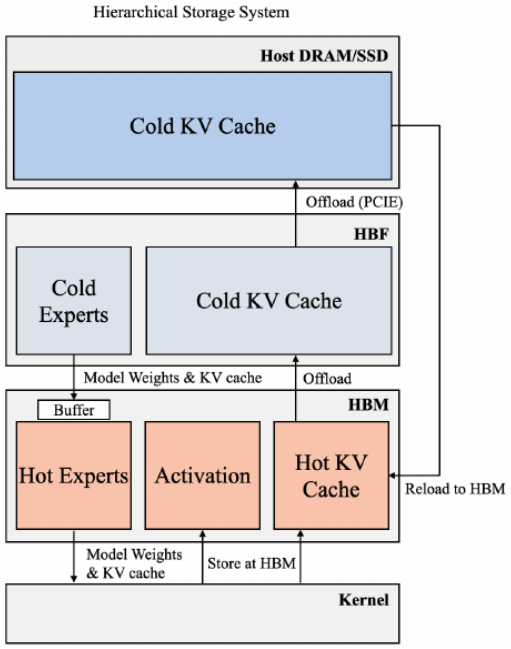
补充知识： HBF 读路径上的硬件延迟隐藏缓冲区，用于提前取数；后文的“10% buffer”是调度器的准入余量，二者用途不同。

二、设计方案

1. 改进机会

- **分层存储：** 频繁访问或需要更新的数据留在 HBM，较冷、占空间的数据向 HBF 和主机迁移。
- **控制并发：** 当前多接一个请求，可能换来后续多次回载和写入。适当留下空间，可以保住即将复用的前缀。
- **利用访问差异：** MoE 热门专家优先放 HBM；稀疏注意力只读少量 KV，但索引器要扫描整个上下文的索引，索引应比 KV 更优先保留在 HBM。[3–4]

2. 系统概览：HBM—HBF—主机分层

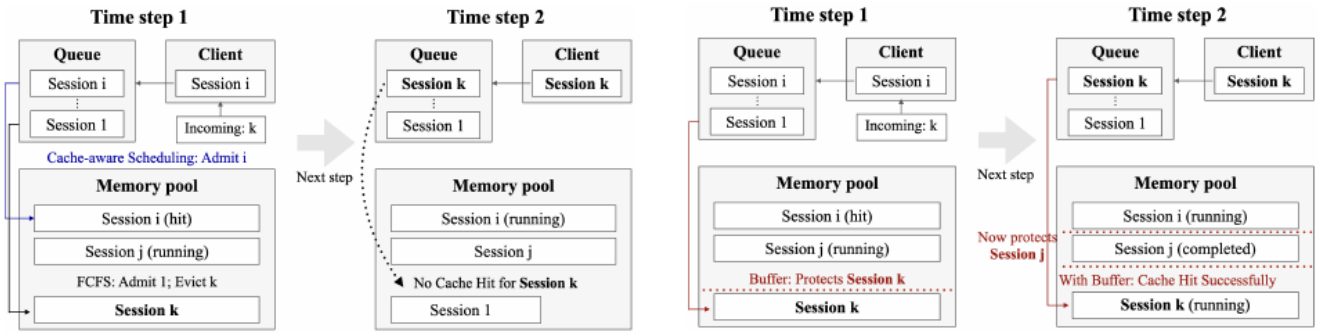


原文图 3：主机包括 DRAM 和 SSD；HBF 读路径中的 Buffer 为 H^3 的硬件延迟隐藏缓冲区。

- **HBM 的部分（热数据与写入入口）：** 新生成的 KV、从主机回载的前缀先进入 HBM，短生命周期激活也写在 HBM。权重可全部放 HBM、全部放 HBF，或把热门专家与其余专家分开放置；每个实验中的权重放置固定。
- **HBF 的部分（设备端容量扩展）：** HBM 容量紧张时，将较冷 KV 下沉到 HBF。按请求级 LRU 选择可迁移缓存；活跃请求受引用锁保护，不参与普通 LRU 驱逐，但其状态仍可在容量允许时从 HBM 移到 HBF。
- **主机的部分（溢出与回载）：** 设备放不下的缓存卸载到主机 DRAM，再溢出到 SSD；SSD 也不足时丢弃。后续请求回载已有前缀，缺失的前缀重新计算。请求完成后，缓存仍可保留等待下一轮复用。
- **索引的部分（index-first）：** HBM 紧张时先迁移 KV，再迁移索引；HBF 紧张时优先将索引卸载到主机，便于下次回载到 HBM。索引与 KV 的驻留位置分开管理。[3–4]

3. 调度方案：带准入余量的缓存感知调度

- **原有方法：** 优先调度设备端已有更多前缀的请求，同分时按队列顺序处理。它关注“谁先运行”，但若把空间塞满，decode 继续增长仍会挤出可复用缓存。
- **本文改进：** 接纳请求前检查 $L+A_i \leqslant bC$。C 是扣除驻留权重后的设备 KV/索引容量，L 是当前活跃请求锁定的数据量，A_i 是接纳该请求后新增锁定的完整上下文数据量，包括原先缓存、现在重新锁定的前缀。
- **10% 余量：** 设 $b=0.9$，限制准入时锁定的活跃数据量，为后续 decode 增长和历史缓存保留空间。这是准入检查，后续 decode 仍可增长；并非每一步都维持一个固定的 10% 物理空闲分区。[4]



原文图 4（左）：接纳新请求挤出 k 的前缀，丢失下一次命中。图 5（右）：留下准入余量，保留前缀供 k 下一轮复用。

补充知识： 少接请求可能降低瞬时并发，却可以减少后续回载、重算与重复写入。这里优化的是整批会话的总完成时间。

三、实验结果

平台： 请求级、轨迹驱动模拟器，采用 B200 计算模型；kernel 延迟由 H100/B200 测量校准。HBM-only 模拟与 H100 上 vLLM 的 16 个吞吐测试点比较，平均绝对百分比误差为 **9.14%**；HBF 性能属于模型外推。

配置： 权重和缓存按 16-bit 存储；每 GPU 基线为 8×24 GB＝**192 GB HBM**，H^3 再加 8×375 GB＝**3 TB HBF**。主机为 1.024 TB DRAM、32 TB SSD，GPU—主机采用 PCIe Gen5 x16。

负载： 来自 LMCache 多轮 Agent 轨迹。Qwen3-32B 用单 GPU、25k 轮；Llama-3.1-405B 用 8 GPU、100k 轮；GLM-5.2 用 16 GPU，比较 100k 轮原始轨迹与 500k 轮合成延长轨迹，后者平均输入由 **22,558** 增至 **89,125 tokens**。[1–2、5]

效果 1：准入余量减少写入，延长估算寿命

Qwen3-32B、H^3 8+8、权重驻留 HBM，固定 25k 轮工作负载：[6]

调度方法	总完成时间（s）	HBF KV 写入（TB）	设备命中率	估算写入寿命
FCFS	13,552.76	106.79	32.84%	1.21 年
缓存感知	10,930.78	21.81	86.87%	4.77 年
缓存感知+10% 余量	10,419.17	6.69	96.46%	14.82 年

- **与 FCFS 相比：** 完成时间减少 **23.1%**，HBF KV 写入减少约 **93.7%**，估算寿命从 **1.21 年增至 14.82 年**。
- **仅看增加余量的收益：** 与无余量缓存感知相比，写入再减少 **69.33%**，完成时间减少 **4.68%**。不能把全部收益都归因于 10% 余量。
- **扩容也可能无益：** 共享接口 4+4 即使启用余量，仍比启用余量的纯 HBM 完成得慢 **1.84%**，能耗也更高。

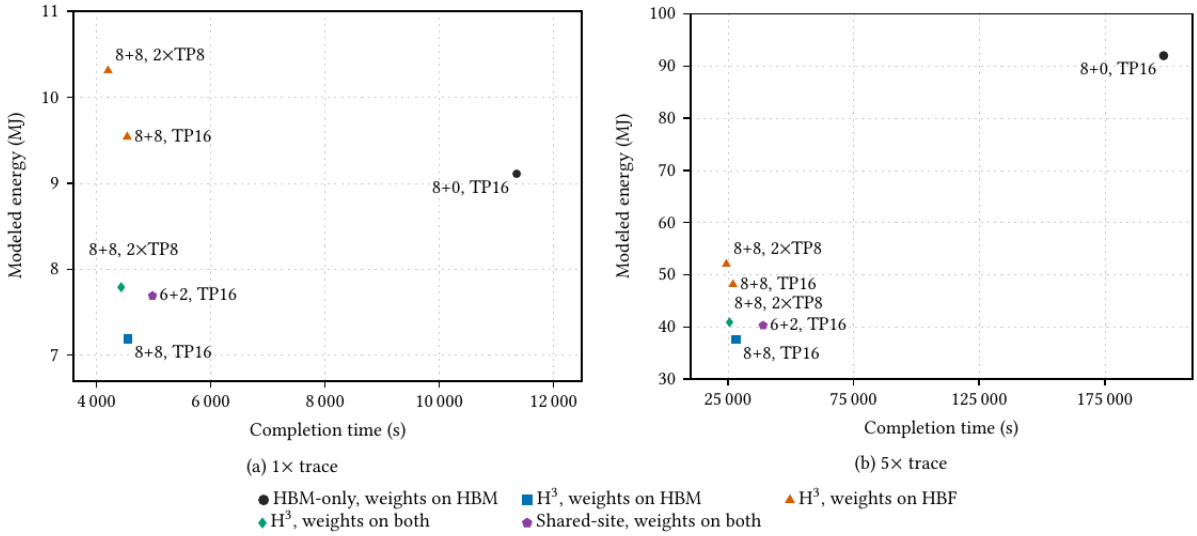
寿命估算采用 **Lwrite = P × C_F ÷ (W_F/T)**：P 为擦写次数，C_F 为 HBF 容量，W_F/T 为平均写入速率。假设 **100,000 次 P/E、均匀磨损、写放大为 1**，并外推当前写入速率；它不是整机寿命或商用 HBF 寿命保证。

效果 2：大模型与长上下文获益更明显

下表均相对同模型、同轨迹、同 GPU 数量的纯 HBM 基线，使用 10% 准入余量：[6–7]

模型与配置	总完成时间	建模能耗	平均 TPOT
Llama 405B，1×，H^3 TP8，HBM 权重	39,666→21,439 s（−45.95%）	53.60→40.70 MJ（−24.06%）	90.85→438.61 ms
GLM-5.2，1×，H^3 TP16，HBM 权重	11,354→4,543 s（−59.99%）	9.11→7.19 MJ（−21.09%）	218.72→279.15 ms
GLM-5.2，5×，H^3 TP16，HBM 权重	198,411→28,279 s（约 7.0× 加速）	91.99→37.62 MJ（−59.10%）	191.93→355.63 ms

- **最快与最省能耗不是同一配置：** GLM 5× 下，两组 TP8 副本、权重全放 HBF 最快，完成时间为 **24,385 s**，比基线减少 **87.7%**；能耗为 **52.06 MJ**。最低能耗 **37.62 MJ** 来自表中的 TP16、HBM 权重配置。
- **权重迁移有代价：** GLM 1×、H^3 TP16 下，把权重从 HBM 移到 HBF，时间几乎不变，能耗却增加 **32.78%**，并超过纯 HBM 基线。



原文图 6：左为 1×，右为 5×；越靠左完成越快，越靠下能耗越低。

效果为什么会有这么大差别

- **更大 batch 摊薄权重读取：** HBF 释放设备端容量，使更多请求共享一次权重加载。GLM 5×、H^3、HBM 权重配置中，仅权重读取减少就贡献 **55.14 个百分点**的基线能耗节省，是总节省 59.10% 的主要来源。
- **历史 KV 留得住，回载和重算就更少：** Llama 1× 避免了 **5,333 万个 query-token** 的重复 prefill 计算，贡献 **20.25 个百分点**的能耗节省；对应 GLM 对照没有重复 prefill，不能用这一原因解释其收益。
- **KV 放 HBF 也会增加读取能耗：** Llama 1× 的 KV/索引读取增加，抵消 **13.16 个百分点**的节能；GLM 使用稀疏注意力，H^3 1× 对应代价仅为 **0.63 个百分点**。
- **寿命收益来自减少缓存搬进搬出：** 调度保留前缀，减少 HBM→HBF 的重复下沉写入，而不是提高 NAND 本身的擦写次数。[6–8]

局限性分析

- **吞吐与延迟有取舍：** 总完成时间缩短的同时，Llama 平均 TPOT 从约 **91 ms 增至 439 ms**；GLM 5× 的所有混合配置 TPOT 都高于纯 HBM。论文不能证明其满足低延迟在线服务要求。
- **硬件与寿命依赖假设：** H^3 假设通过 HBM 缓冲充分隐藏 HBF 读取延迟；没有完整生产级 HBF 控制器、磨损均衡及所有激活/暂存写入模型。大 batch 与 HBF 参数仍需实物验证。
- **能耗口径有限：** 按运算次数、存储访问和通信流量计能耗，**不计静态功耗和完整封装功耗**。运行时间缩短本身不会直接降低该模型中的能耗；H^3 使用更多堆栈，也没有等采购成本或 TCO 对照。
- **负载与策略外推有限：** 首轮请求均在零时刻提交，后续按会话依赖和轮间间隔到达；5× 负载为拼接构造。MoE 放置依赖离线画像并假设理想专家负载均衡，10% 余量的最优性未得到普遍证明。[2–3、5、8]

来源定位（本地 PDF 页码）：[1] p.6 表1、§5.1；[2] p.2–4 §3；[3] p.4 §4.1–4.2；[4] p.4–6 §4.2–4.3、图3–5；[5] p.6 §5.1、表3；[6] p.7 §5.2、表4–5、式(8)；[7] p.7–8 §5.3、表6、图6；[8] p.9 §5.4–5.5、表7。原因解释与局限性分析依据这些章节整理，非新增实验。